ESP-NOW Protocol Tutorial

Direct ESP32-to-ESP32 Wireless Communication using Arduino IDE

Prerequisites & Hardware

- Hardware: One or more ESP32 development boards. For the basic two-device test, use two ESP32 boards.
- USB data cable: Required for programming and Serial Monitor access. Make sure the cable supports data, not only charging.
- Power: USB power is sufficient for normal testing. A separate regulated supply can be used if the board's power-input requirements are satisfied.
- Software: Arduino IDE installed on your computer.

What is ESP-NOW?

ESP-NOW is Espressif's connectionless communication protocol for direct communication between ESP devices. It uses the Wi-Fi radio but does not require the devices to associate with a normal Wi-Fi access point. It is intended for low-latency, low-power local communication such as sensors, remote controls and smart devices.

ESP-NOW supports direct unicast communication, broadcast communication, and larger network arrangements such as one-to-many and many-to-many. The same device can act as a sender and receiver.

Item	Details
Communication	Direct device-to-device communication; no access point required.
Addressing	ESP-NOW unicast uses the peer's MAC address.
Data	Application data is transmitted as a byte buffer.
Use cases	Sensors, remote controls, local device control, low-latency IoT links.

Espressif describes ESP-NOW as a connectionless Wi-Fi communication protocol in which application data is carried in vendor-specific action frames. The default ESP-NOW PHY rate is 1 Mbps. Current ESP-NOW documentation describes v1.0 and v2.0; for broad compatibility, keeping an application payload within the older 250-byte limit is a practical beginner rule.

Step 1: Configure the Arduino IDE

1. Install ESP32 Board Support

1. Open Arduino IDE.
2. Go to File > Preferences.
3. Add the Espressif ESP32 Boards Manager URL to Additional Boards Manager URLs if it is not already present.
4. Go to Tools > Board > Boards Manager.
5. Search for esp32 by Espressif Systems and click Install.
6. Connect the ESP32 to the computer with a USB data cable.

7. Select the appropriate ESP32 board under Tools > Board and select the correct COM port under Tools > Port.

No separate ESP-NOW library needs to be installed for the classic ESP32 Arduino API used in this tutorial. The ESP-NOW API is provided with the ESP32 Arduino core.

NOTE: The exact board name shown in Arduino IDE can vary with the ESP32 variant. Select the board definition that matches the module you are actually using.

Step 2: Find the MAC Address of Each ESP32

For unicast communication, the sender needs the MAC address of the receiver. Run the following sketch on each ESP32 separately and record the printed address.

```
#include <WiFi.h>
#include "esp_mac.h"

void setup() {
  Serial.begin(115200);
  delay(1000);

  WiFi.mode(WIFI_STA);

  uint8_t mac[6];
  esp_read_mac(mac, ESP_MAC_WIFI_STA);

  Serial.printf("MAC Address: %02X:%02X:%02X:%02X:%02X:%02X\n",
                mac[0], mac[1], mac[2],
                mac[3], mac[4], mac[5]);
}

void loop() {
}
```

Run and verify

8. Upload the sketch to ESP32-A.
9. Open Tools > Serial Monitor and set the baud rate to 115200.
10. Record the MAC address.
11. Repeat the process for ESP32-B.

```
Example:
ESP32-A MAC: 24:6F:28:AA:BB:CC
ESP32-B MAC: 14:08:08:A4:61:A0
```

NOTE: The addresses above are examples. Always use the actual MAC addresses printed by your own boards.

Step 3: Configure the ESP-NOW Receiver

The receiver initializes ESP-NOW and registers a receive callback. The callback is automatically invoked when an ESP-NOW packet arrives.

```
#include <WiFi.h>
#include <esp_now.h>

#define LED_PIN 2

void onReceive(const esp_now_recv_info_t *info,
               const uint8_t *data,
               int len) {

  Serial.print("Received: ");

  for (int i = 0; i < len; i++) {
    Serial.print((char)data[i]);
  }

  Serial.println();

  digitalWrite(LED_PIN, HIGH);
  delay(200);
  digitalWrite(LED_PIN, LOW);
}

void setup() {
  Serial.begin(115200);
  delay(1000);

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  WiFi.mode(WIFI_STA);
  WiFi.setChannel(1);

  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW initialization failed");
    return;
  }

  esp_now_register_recv_cb(onReceive);

  Serial.println("ESP-NOW Receiver Ready");
}

void loop() {
}
```

What the receiver code does

- WiFi.mode(WIFI_STA) enables the ESP32 station interface.
- WiFi.setChannel(1) places the radio on Channel 1.
- esp_now_init() initializes ESP-NOW.

- esp_now_register_recv_cb(onReceive) tells ESP-NOW which function to call when a packet is received.
- The callback reads the received byte buffer and prints it.
- The LED blinks when the receive callback executes.

Step 4: Configure the ESP-NOW Sender

For unicast, the sender registers the receiver as a peer and sends data to the receiver's MAC address.

```
#include <WiFi.h>
#include <esp_now.h>

#define LED_PIN 2

// Replace this with the receiver's actual MAC address
uint8_t receiver[] = {
  0x14, 0x08, 0x08,
  0xA4, 0x61, 0xA0
};

void setup() {
  Serial.begin(115200);
  delay(1000);

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  WiFi.mode(WIFI_STA);
  WiFi.setChannel(1);

  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW INIT FAILED");
    return;
  }

  esp_now_peer_info_t peerInfo = {};

  memcpy(peerInfo.peer_addr, receiver, 6);
  peerInfo.channel = 1;
  peerInfo.encrypt = false;

  if (esp_now_add_peer(&peerInfo) != ESP_OK) {
    Serial.println("PEER ADD FAILED");
    return;
  }

  Serial.println("Unicast Sender Ready");
}

void loop() {
  const char *message = "Hello from ESP32-A";

  esp_err_t result = esp_now_send(
    receiver,
    (uint8_t *)message,
    strlen(message)
  );
```

```
  if (result == ESP_OK) {
    Serial.println("Message sent");

    digitalWrite(LED_PIN, HIGH);
    delay(200);
    digitalWrite(LED_PIN, LOW);
  } else {
    Serial.println("Send failed");
  }

  delay(2000);
}
```

Understanding the peer

A peer is an ESP-NOW device that is registered for communication. The peer information contains the target MAC address, communication channel and encryption setting.

```
esp_now_peer_info_t peerInfo = {};

memcpy(peerInfo.peer_addr, receiver, 6);
peerInfo.channel = 1;
peerInfo.encrypt = false;

esp_now_add_peer(&peerInfo);
```

Step 5: Run the Trial and Verify

12. Upload the receiver sketch to ESP32-B.
13. Open its Serial Monitor at 115200 baud. It should display: ESP-NOW Receiver Ready.
14. Upload the sender sketch to ESP32-A.
15. The sender should display: Unicast Sender Ready followed by Message sent every 2 seconds.
16. The receiver should display: Received: Hello from ESP32-A every 2 seconds.
17. The sender LED indicates that esp_now_send() returned ESP_OK. The receiver LED indicates that the receive callback executed.

Expected output

```
Sender:
Unicast Sender Ready
Message sent
Message sent
Message sent

Receiver:
ESP-NOW Receiver Ready
Received: Hello from ESP32-A
Received: Hello from ESP32-A
Received: Hello from ESP32-A
```

NOTE: ESP_OK from esp_now_send() means the send API call succeeded. It should not be interpreted as proof that the receiver's application code processed the packet. A send callback can be added later to monitor transmission status.

Step 6: Send an Information / Data Packet

Instead of sending a text string, ESP-NOW can carry application data as bytes. A convenient approach for sensor systems is to define the same structure on both ESP32s and transmit the structure as a byte buffer.

```
struct SensorData {
  float temperature;
  float voltage;
  float current;
  float battery;
};
```

The sender fills the fields, then sends the memory occupied by the structure. The receiver checks the packet length and copies the received bytes back into its own SensorData structure.

6.1 Sender — Structured Data

```
#include <WiFi.h>
#include <esp_now.h>

#define LED_PIN 2

uint8_t receiver[] = {
  0x14, 0x08, 0x08,
  0xA4, 0x61, 0xA0
};

struct SensorData {
  float temperature;
  float voltage;
  float current;
  float battery;
};

SensorData data;

void setup() {
  Serial.begin(115200);
  delay(1000);

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  WiFi.mode(WIFI_STA);
  WiFi.setChannel(1);

  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW INIT FAILED");
    return;
  }

  esp_now_peer_info_t peerInfo = {};
  memcpy(peerInfo.peer_addr, receiver, 6);
  peerInfo.channel = 1;
  peerInfo.encrypt = false;
```

```
  if (esp_now_add_peer(&peerInfo) != ESP_OK) {
    Serial.println("PEER ADD FAILED");
    return;
  }

  Serial.println("Sender Ready");
}

void loop() {
  data.temperature = 28.5;
  data.voltage = 12.4;
  data.current = 1.8;
  data.battery = 87.0;

  Serial.print("Sending bytes: ");
  Serial.println(sizeof(data));

  esp_err_t result = esp_now_send(
    receiver,
    (uint8_t *)&data,
    sizeof(data)
  );

  if (result == ESP_OK) {
    Serial.println("Packet sent");

    digitalWrite(LED_PIN, HIGH);
    delay(200);
    digitalWrite(LED_PIN, LOW);
  } else {
    Serial.println("Send failed");
  }

  delay(2000);
}
```

6.2 Receiver — Structured Data

```
#include <WiFi.h>
#include <esp_now.h>

#define LED_PIN 2

struct SensorData {
  float temperature;
  float voltage;
  float current;
  float battery;
};

SensorData data;

void onReceive(const esp_now_recv_info_t *info,
               const uint8_t *incomingData,
               int len) {
```

```
  Serial.print("Received bytes: ");
  Serial.println(len);

  Serial.print("Expected bytes: ");
  Serial.println(sizeof(SensorData));

  if (len != sizeof(SensorData)) {
    Serial.println("Invalid packet size");
    return;
  }

  memcpy(&data, incomingData, sizeof(data));

  Serial.println("----- RECEIVED DATA -----");

  Serial.print("Temperature: ");
  Serial.print(data.temperature);
  Serial.println(" C");

  Serial.print("Voltage: ");
  Serial.print(data.voltage);
  Serial.println(" V");

  Serial.print("Current: ");
  Serial.print(data.current);
  Serial.println(" A");

  Serial.print("Battery: ");
  Serial.print(data.battery);
  Serial.println(" %");

  Serial.println("-------------------------");

  digitalWrite(LED_PIN, HIGH);
  delay(200);
  digitalWrite(LED_PIN, LOW);
}

void setup() {
  Serial.begin(115200);
  delay(1000);

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  WiFi.mode(WIFI_STA);
  WiFi.setChannel(1);

  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW initialization failed");
    return;
  }

  esp_now_register_recv_cb(onReceive);

  Serial.println("ESP-NOW Receiver Ready");
}
```

```
void loop() {
}
```

Step 7: Understand the Data Packet

```
Sender:

SensorData
+----------------------+
| temperature          |
| voltage              |
| current              |
| battery              |
+----------+-----------+
           |
           | raw bytes
           v
       ESP-NOW
           |
           v
Receiver:
incomingData
           |
           | memcpy()
           v
       SensorData
```

Step 8: Broadcast Communication (Optional)

Broadcast sends a packet to all ESP-NOW devices listening on the appropriate channel rather than to one specific MAC address.

```
uint8_t broadcastAddress[] = {
  0xFF, 0xFF, 0xFF,
  0xFF, 0xFF, 0xFF
};
```

A specific MAC address is unicast. FF:FF:FF:FF:FF:FF is the broadcast destination. Do not name a variable broadcastAddress and then put a specific peer MAC inside it; the destination value determines the actual addressing mode.

Troubleshooting

1. Receiver only prints 'ESP-NOW Receiver Ready'

- Verify both ESP32s are powered and running.
- Verify both devices are configured to the same Wi-Fi channel.
- Verify the sender uses the receiver's actual MAC address for unicast.
- Check that esp_now_add_peer() does not report PEER ADD FAILED.
- Confirm that the receiver registered esp_now_register_recv_cb(onReceive).
- If the sender uses broadcast, confirm the destination is FF:FF:FF:FF:FF:FF.

2. MAC address appears as 00:00:00:00:00:00

- Use the esp_read_mac() sketch in Step 2 to read the Wi-Fi station MAC.
- Check that WiFi.mode(WIFI_STA) is set before reading the station MAC.
- Do not copy an example MAC address; use the address printed by the actual board.

3. 'Invalid packet size' appears

- Make sure both sketches define the same SensorData structure.
- Make sure the sender is sending (uint8_t *)&data with sizeof(data).
- Check the printed Received bytes and Expected bytes values.
- For the example four-float structure, both should normally be 16.

4. Sender says 'Message sent' but receiver gets nothing

- Remember that ESP_OK from the immediate send call is not the same as application-level confirmation.
- Check the receiver MAC and channel first.
- Then add the ESP-NOW send callback to inspect transmission status.
- Avoid long blocking work inside ESP-NOW callbacks in a production application; Espressif documents that these callbacks run in the Wi-Fi task.

ESP-NOW Quick Reference

Item	Details
Initialize	esp_now_init()
Set Wi-Fi mode	WiFi.mode(WIFI_STA)
Set channel	WiFi.setChannel(CHANNEL)
Register receiver	esp_now_register_recv_cb(onReceive)
Define peer	esp_now_peer_info_t peerInfo
Add peer	esp_now_add_peer(&peerInfo)
Send data	esp_now_send(destinationMAC, data, length)
Unicast destination	Receiver's actual 6-byte MAC address
Broadcast destination	FF:FF:FF:FF:FF:FF
Serial Monitor	115200 baud

Recommended Next Steps

18. Add the ESP-NOW send callback and understand delivery status.
19. Implement bidirectional ESP32-to-ESP32 communication.
20. Send a custom packet containing sensor values, device ID, sequence number and timestamp.
21. Implement acknowledgements at the application level when required.
22. Test one-to-many communication and broadcast.
23. Learn ESP-NOW encryption and peer security.
24. Measure range, latency and packet loss.
25. Integrate real sensors and build the final IoT application.

References

Espressif ESP-NOW Programming Guide:

https://docs.espressif.com/projects/esp-now/en/latest/esp32/

Espressif Arduino-ESP32 ESP-NOW API:

https://docs.espressif.com/projects/arduino-esp32/en/latest/api/espnow.html

Espressif ESP-IDF ESP-NOW API:

https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html